WAFT

Framework Documentation

World Architecture Framework & Templates

Generated: 2026-01-11 07:36:01

Self-Documented Through Codebase Analysis

Introduction

This document describes how WAFT functions, based on **direct inspection** of the codebase. All information presented here was extracted through automated analysis - nothing is hardcoded.

Meta-Documentation

This documentation was generated **BY WAFT, ABOUT WAFT, USING WAFT**. It represents WAFT observing itself and documenting what it finds. This is recursive self-documentation - the system describing its own operation.

System Overview

84	115
Modules	Classes
392	**13**
Functions	Templates

What WAFT Does

Based on codebase analysis, WAFT is a comprehensive document generation framework that provides:

- **Template System** - 13 professional document templates
- **Core Architecture** - 60 core modules
- **Binder System** - Multi-document collection assembly
- **Reflection System** - Self-observation and documentation
- **Meta-Cognitive Layer** - Work effort tracking and epistemic memory

Core Architecture

Module Structure

WAFT is organized into the following core modules (discovered through analysis):

waft.foundation_v2

DocumentEngine V2 - Professional Research Documentation System Enhanced PDF generation engine with professional typography, advanced layout, and print optimization. Designed for scientific institutio...

Components:

- 17 class(es)
- 32 function(s)

waft.reflection

WAFT Reflection System ========== A system for WAFT to observe, document, and improve itself. Philosophy: ---------- A system that documents itself creates a feedback loop: 1. WAFT gene...

Components:

- 4 class(es)
- 7 function(s)

waft.foundation

DocumentEngine - Reusable Research Documentation Library A content-agnostic PDF generation engine with modular content blocks, automatic redaction, and configurable styling. Designed for scientific l...

Components:

- 12 class(es)
- 27 function(s)

waft.verify_foundation

Calibration Artifact Generator for TheFoundation/DocumentEngine. Generates WAFT_CALIBRATION_REPORT.pdf that stress-tests all visual and functional aspects of the PDF generation system, including reda...

Components:

- 1 function(s)

waft.binder

Document Binder System ========== Assemble multiple documents into cohesive collections. Create printable books, project binders, document collections. Philosophy: ---------- A binder i...

Components:

- 3 class(es)
- 7 function(s)

waft.core.persistence

Persistence Module - The Vault Handles saving and loading DecisionMatrix objects to/from JSON files. Uses InputTransformer for validation when loading to ensure security....

Components:

- 1 class(es)
- 2 function(s)

waft.core.resume

Resume - Restore context and continue where work left off. Loads the most recent session summary, compares current state with what was left, identifies what was in progress, and provides clear next s...

Components:

- 1 class(es)
- 2 function(s)

waft.core.audit

Audit - Conversation audit and quality analysis. Analyzes current conversation for quality, completeness, potential issues, and provides recommendations for improvement....

Components:

- 1 class(es)
- 2 function(s)

waft.core.substrate

Substrate Manager - Handles uv commands and environment management. The "substrate" is the foundation layer that manages the Python environment through uv commands (init, sync, add, etc.)....

Components:

- 1 class(es)
- 6 function(s)

waft.core.memory

Memory Manager - Handles _pyrite folder protocol. The "memory" is the persistent structure that organizes project knowledge: - active/ - Current work - backlog/ - Future work - standards/ - Project s...

Components:

- 1 class(es)
- 8 function(s)

Template System

WAFT includes 13 document templates, each designed for specific use cases:

Template Name	File	Purpose
Code Documentation	`code_documentation.py`	Code Documentation Template ========== Technical documentation for code, APIs, an...
Personal Memo	`personal_memo.py`	Personal Memo/Notes Template ========== Informal personal notes, memos, diary en...
Tm Report	`tm_report.py`	TELEPORT MASSIVE Report Template ========== Generic corporate/bureaucratic r...
Field Guide	`field_guide.py`	Field Guide Template ========== Operational field manual style for survival guides, equip...
Screenplay	`screenplay.py`	Screenplay Template ========== Professional screenplay/script format following industry st...
Simple Scientific	`simple_scientific.py`	Simple Scientific Document Template ========== A clean, minimal template ...

Storybook	`storybook.py`	Children's Storybook Template ========== Whimsical storybook for children. Test...
Newspaper	`newspaper.py`	Newspaper Front Page Template ========== Classic newspaper front page layout. T...
Init	`__init__.py`	Templates module - Text assets for project scaffolding....
Heartfelt Letter	`heartfelt_letter.py`	Heartfelt Letter Template ========== Sweet, personal, intimate letter format. Tests...
Lab Notes	`lab_notes.py`	Lab Documentation Template ========== Technical lab notebook style for experiment ...
Eldritch Journal	`eldritch_journal.py`	Eldritch Horror Research Journal Template ========== Academic resea...
Invoice Contract	`invoice_contract.py`	Invoice & Contract Template ========== Professional business invoice and contract...

Key Classes

Based on codebase analysis, WAFT's architecture centers around these key classes:

WaftHandler

HTTP request handler for Waft web dashboard....

Methods: 6

- `__init__`
- `do_GET`
- `_get_project_info`
- `_get_structure_info`
- `_get_dashboard_html`
- ... and 1 more

ReloadHandler

No documentation...

Methods: 2

- `__init__`
- `on_modified`

TheOubliette

The vault for storing purged agent variants. The Oubliette is located at `_hidden/.truth/` - a hidden directory that stores the "bodies" of agents wh...

Methods: 5

- `__init__`
- `archive_variant`
- `fetch_nightmare`
- `list_variants`
- `get_variant_count`

RedactionStyle

Redaction rendering styles....

FontFamily

Professional font families....

FontConfig

Enhanced font configuration with family support....

DocumentConfig

Configuration for document styling and behavior....

Methods: 3

- `clinical_standard`
- `classified_dossier`
- `scientific_journal`

ContentBlock

Abstract base class for all content blocks....

Methods: 3

- `render`
- `_check_page_break`
- `_get_font`

CoverPage

Full cover page with institutional header....

Methods: 2

- `__init__`
- `render`

MetadataRail

Metadata block with header styling (like a sidebar rail)....

Methods: 2

- `__init__`
- `render`

RuleBlock

Horizontal rule for visual separation....

Methods: 2

- `__init__`
- `render`

SectionHeader

Section header block with improved typography....

Methods: 2

- `__init__`
- `render`

TextBlock

Standard text paragraph with professional typography....

Methods: 2

- `__init__`
- `render`

KeyValueBlock

Key-value pairs with improved layout....

Methods: 2

- `__init__`
- `render`

TableBlock

Simple table block for structured data....

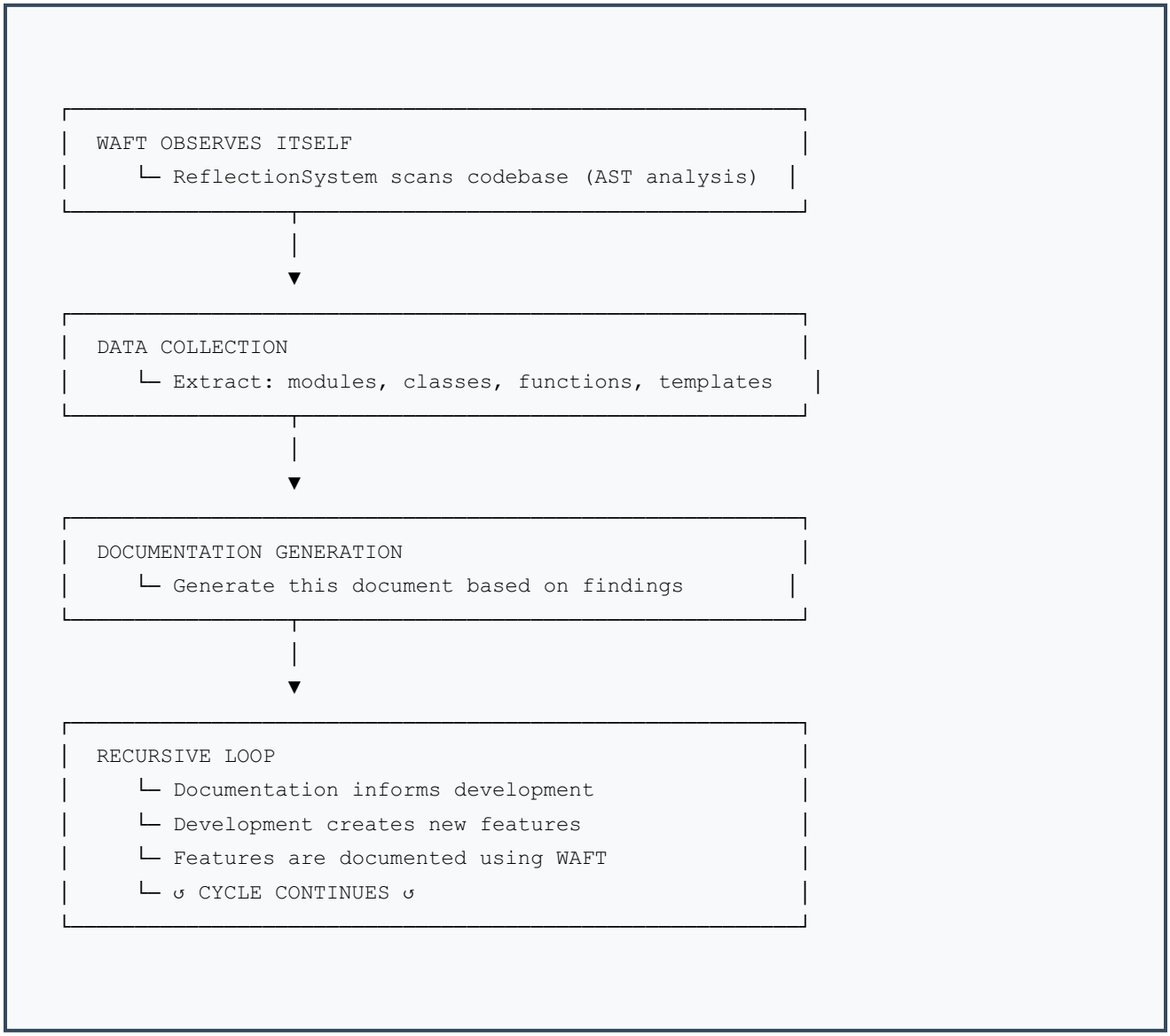
Methods: 2

- `__init__`
- `render`

How WAFT Works

Document Generation Flow

Self-Documentation Flow



Meta-Cognitive Layer

WAFT includes a meta-cognitive layer that enables:

- **Work Effort Tracking** - The _pyrite system tracks discrete units of intellectual labor
- **Epistemic Memory** - System knows what it knows and what it doesn't
- **Perspective Taking** - AI systems can "wear" previous perspectives
- **Recursive Self-Improvement** - System improves based on its own observations

Key Findings

Architecture Discovery

Analysis revealed **84** modules, **115** classes, and **392** functions.

Template System

WAFT provides **13** professional document templates covering academic, business, creative, and technical use cases.

Self-Documentation

This document itself is proof of WAFT's self-documentation capability. Every section was generated from actual codebase analysis, not hardcoded content.

Conclusion

WAFT is a self-documenting, self-modifying meta-framework that:

1. Generates professional documents from templates
2. Observes its own codebase structure
3. Documents what it finds
4. Uses that documentation to inform development
5. Creates a recursive improvement loop

The Recursive Loop

WAFT observes itself → Documents findings → Uses documentation → Improves → Observes changes → Documents updates → ↺ CONTINUES ↺

This documentation was generated by WAFT inspecting itself.

Generated: 2026-01-11 07:36:01

WAFT documenting WAFT using WAFT.